tool-latency-bench

FINDINGS · COMPACT VERSION

Reading 60 tool definitions adds almost no latency on Gemini 3.1 Flash-Lite

Pradnesh Fernandez A · 6 November 2026 · ~1,250 streamed requests, Gemini API (OpenAI-compatible endpoint, paid tier) · all intervals 95%

820 ms
median time to first answer, no tools

+7 ms
per 1,000 tool tokens, read but not called [−12, +27]

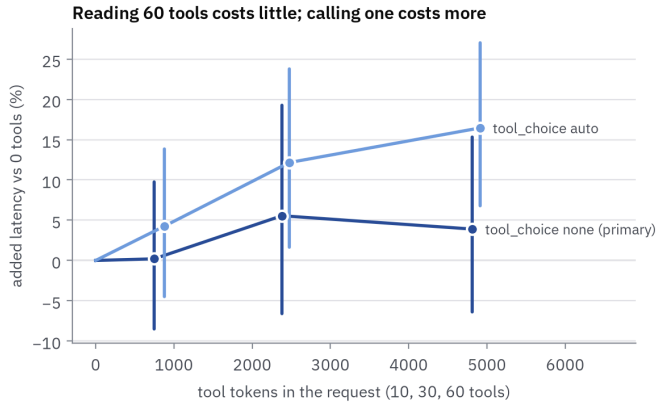
+16%
at 60 tools when the model may call them [+7, +27]

≈40×
larger: switching to Gemini 3.8 Flash vs adding 60 tools

Setup in brief. The same user message is sent with 0 / 10 / 30 / 60 real tool definitions from open-source voice agents (exp1), or with 20 simple / 20 medium / 10+10 mixed tools at a fixed ~3,060-token budget (exp2), in random order within paired blocks, with a nonce against caching. `tool_choice: none` measures the cost of reading the tools; `auto` lets the model call them. Reasoning off, 2 sessions × 30 blocks per experiment and mode, standard errors clustered on the user message. Metric: time to first answer text, or to the completed tool call.

Gemini 3.1 Flash-Lite, pooled	`tool_choice none`	`auto`
Added latency per 1,000 tool tokens	`+7 ms [−12, +27]`	`+27 ms [+10, +44]`
60 tools (~4,900 tokens) vs 0	`+3.9% [−6, +15]`	`+16.5% [+7, +27]`
Medium vs simple tools, same budget	`+4% [−8, +18]`	`+8% [−1, +18]`
Median latency, 0 → 60 tools	`820 → 837 ms`	`895 → 1,080 ms`

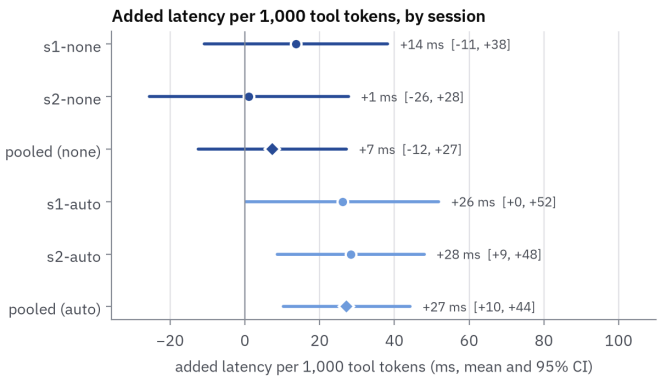
1. Reading tools is cheap



exp1, 2 sessions pooled: change vs the same block's 0-tool request.

- Slope **+7.4 ms per 1,000 tool tokens [−12.5, +27.2]**.
- 60 tools: **+3.9% [−6.4, +15.3]**. At the top of the interval that is ~130 ms; the best estimate is ~35 ms.
- Both sessions agree: s1 +13.7, s2 +1.1 ms per 1k (chart below).

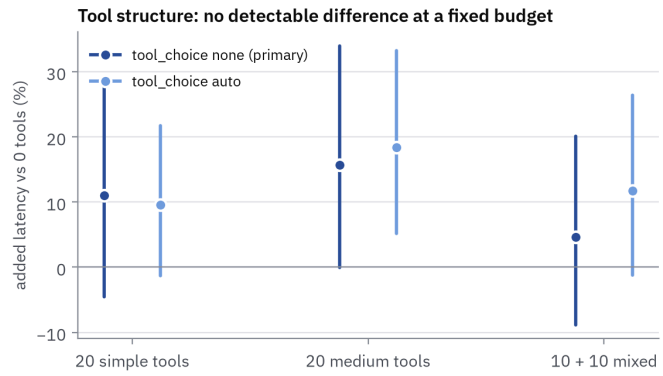
2. Letting the model call tools costs more



Slope per 1,000 tool tokens, per session and pooled, both modes.

- With `auto`: **+27.2 ms per 1k [+10.2, +44.2]**; 60 tools **+16.5% [+6.8, +27.0]**. Sessions agree (+26.1, +28.3).
- The model calls a tool in 58% / 87% / 93% of requests at 10 / 30 / 60 tools.
- Most of the gap between the modes is the time to emit a complete tool call, not to read the list.

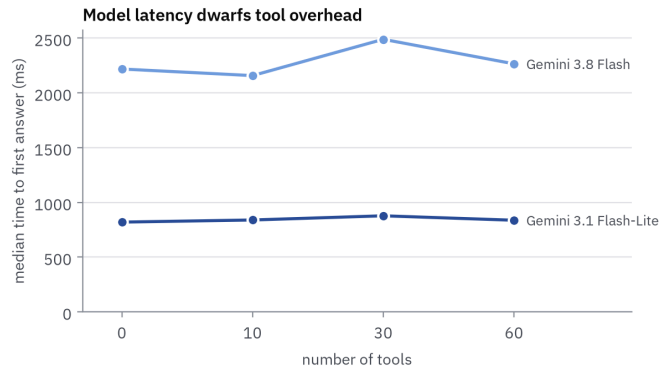
3. Tool structure: no detectable difference



exp2, 2 sessions pooled: change vs 0 tools per condition.

- Medium vs simple at the same budget: **+4.3% [−8.1, +18.3]**; per-1k-token difference +9.9 ms [−48.7, +68.4].
- Mixed vs simple −5.7% [−14.8, +4.3]. Sessions disagree on the sign: noise around zero.
- Gemini counts the medium sets as **+13.8%** more tokens than tiktoken does, so the per-token difference is the fair comparison.

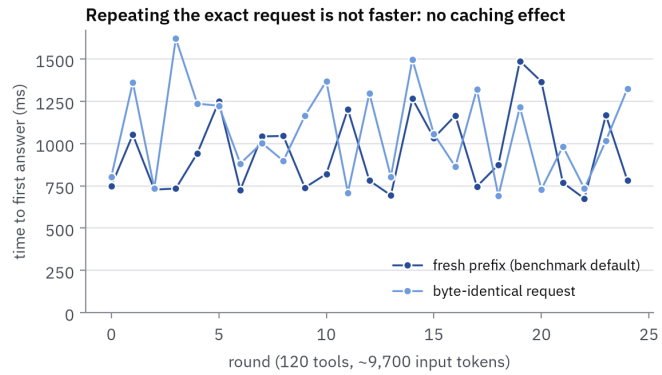
4. The model matters far more than the tools



Median latency by tool count, `tool_choice: none`, one session per model.

- Gemini 3.8 Flash: **~2.2 s** at every tool count vs 0.82 s for Flash-Lite. A ~1.4 s gap, about 40× the cost of 60 tools.
- `minimal` is rejected (HTTP 400) and `none` doesn't reliably stop thinking when tools are present: empty replies in 2/120 (`none`) and 20/120 (`auto`). Reported as a finding, not a tool-overhead estimate.

5. Checks



Caching probe: 120 tools, 25 rounds, identical vs fresh request.

- **No caching effect:** ratio 1.11 [0.96, 1.29]. Gemini reports no cached tokens, so this probe is the evidence.
- **Hallucinated calls:** Flash-Lite invents `get_reward_points` with *no* tools declared (3 of 6 tries for one message). Excluding that message changes nothing.
- Order within a block, drift (824 vs 853 ms) and noise (CV ~0.3) checked; none biases the results.

Limitations. One provider, two sessions on the same afternoon, the OpenAI-compatible endpoint (not the native API Pipecat uses), short prompts, ≤60 tools. **Next:** more days, OpenAI, Groq/Cerebras, 120 tools, the native Gemini API.

> **Takeaway for voice-agent builders:** on a fast model, don't trim your tool list to save latency. Pick the model and keep reasoning off. Tool calls themselves are the cost worth designing around.

Full method and resources: the in-depth report · code: github.com/pradneshfernandez/tool-latency-bench